AGENTISCHE SOFTWAREENTWICKLUNG · TOKEN-EFFIZIENZ

21 agnostische Methoden, um Tokens zu sparen

Eingeordnet nach dem Moment im Agenten-Zyklus, in dem sie wirken: vom Plan über den Kontext und die Tool-Ausführung bis zum Verlauf, der in jedem Turn erneut gesendet wird.

WIRKT AUF 🟦 Input-Tokens 🟧 Output-Tokens 🟩 Preis pro Token 🟪 Iterationen / Aufrufe

TOP-HEBEL größte Wirkung in langen Sessions

PHASE 0

Planen

Wie viele Anläufe braucht die Aufgabe?

20 ITERATIONEN **TOP-HEBEL**

Spec-first / Plan-then-Execute

PROCESS LAYER

Erst einen Plan erstellen und abnehmen, dann umsetzen. Jede Fehlschleife sendet den gesamten Kontext erneut.

Weniger Iterationen schlagen jede Mikro-Kompression

↓

PHASE 1

Aufruf konfigurieren

Welches Modell, wie viel Denken, wann – und braucht es den Aufruf überhaupt?

11 PREIS PRO TOKEN

Model Routing / Cascading

ROUTING LAYER

Günstiges Modell für Klassifikation und Routine-Edits; Eskalation zum teuren Modell nur bei Unsicherheit oder Fehlschlag.

40–70 % geringere Kosten

17 OUTPUT-TOKENS

Reasoning-Budget-Steuerung

MODEL LAYER

Thinking- bzw. Effort-Parameter pro Aufgabe setzen: niedrig für Routine, hoch für Architekturfragen.

Reasoning übersteigt die sichtbare Antwort oft um ein Vielfaches

18 PREIS PRO TOKEN

Batch APIs

INFRASTRUCTURE LAYER

Nicht zeitkritische Jobs asynchron einreichen: nächtliche Reviews, Doku-Generierung, Massenverarbeitung.

rund 50 % Rabatt bei allen großen Anbietern

16 AUFRUFE

Semantic Response Caching

INFRASTRUCTURE LAYER

Antworten auf semantisch ähnliche Anfragen per Embedding-Ähnlichkeit wiederverwenden, etwa mit GPTCache oder Redis Vector.

Aufruf entfällt ganz – bei Coding-Agenten selten nutzbar

↓

PHASE 2

Kontext zusammenbauen

Was geht in den Prompt – und in welcher Form?

02 PREIS PRO TOKEN

Prefix Alignment & Prompt Caching

INFRASTRUCTURE LAYER

Statisches vorne (System-Prompt, AGENTS.md, Tools), Dynamisches hinten. Bei Anthropic mit expliziten Breakpoints, TTL standardmäßig 5 Minuten.

bis zu 90 % günstiger bei Cache-Hits

13 INPUT-TOKENS **TOP-HEBEL**

Dynamic Tool Loading

PROTOCOL LAYER

Nur einen Tool-Index laden, vollständige Schemas bei Bedarf nachladen; ungenutzte MCP-Server pro Projekt deaktivieren.

oft mehrere 10k Tokens pro Call

06 INPUT-TOKENS

Progressive Disclosure / JIT Retrieval

DATA LAYER

Doku und Architekturwissen erst bei erkanntem Bedarf nachladen, etwa über MCP.

kein Vorab-Ballast im System-Prompt

05 INPUT-TOKENS

AST-basiertes Repository Mapping

CONTEXT LAYER

Tree-Sitter plus PageRank erzeugt eine kompakte Symbolkarte statt roher Quelltextdateien.

70–80 % weniger Kontext bei Repo-Analyse

15 INPUT-TOKENS

Token-effiziente Datenformate

INPUT LAYER

Tabellarische Daten als CSV, YAML oder TOON statt als formatiertes JSON in den Kontext geben.

30–60 % bei einheitlichen Objekt-Arrays

14 INPUT-TOKENS

Prompt Compression

INPUT LAYER

LLMLingua entfernt informationsarme Tokens anhand der Perplexity eines kleinen Modells. Gut für RAG, riskant für Code.

wirksam bei Fließtext, nicht bei Syntax

↓

PHASE 3

Inferenz & Antwort

Was gibt das Modell aus?

03 OUTPUT-TOKENS

Caveman Pattern

OUTPUT / MODEL LAYER

System-Prompt weist an, Füllwörter, Höflichkeitsfloskeln und redundante Begründungen zu streichen.

65–75 % weniger Prosa – Gesamteffekt kleiner

07 OUTPUT-TOKENS

Strict Response Schemas

PROTOCOL LAYER

Strukturierte Antworten (JSON, Pydantic, YAML) erzwingen. Ein Reasoning-Feld zuerst ins Schema, sonst leidet die Qualität.

nur das notwendige Datenformat

12 OUTPUT-TOKENS

Diff-basierte Edits

OUTPUT LAYER

Search/Replace-Blöcke oder Unified Diffs statt einer kompletten Neuausgabe der Datei.

90 % und mehr bei großen Dateien

↓

PHASE 4

Tool-Ausführung

Was fließt aus Shell, Dateien und Sub-Agenten zurück?

08 INPUT-TOKENS **TOP-HEBEL**

Sub-Agent-Isolation

ORCHESTRATION LAYER

Codesuche, Log-Analyse und Recherche an Sub-Agenten mit frischem Kontext delegieren. Zurück kommt nur das Ergebnis.

oft der größte Einzelhebel

01 INPUT-TOKENS

RTK (Rust Token Killer)

EXECUTION LAYER

Fängt Ausgaben von git, pytest, cargo test oder docker logs ab und entfernt Boilerplate, bevor sie im Kontext landen.

60–90 % bei Shell- und Tool-Outputs

04 INPUT-TOKENS

File Piping & Stream Filtering

OS LAYER

Große Logs und Dateien nicht nativ einlesen, sondern mit grep, head und awk vorfiltern.

verhindert Tausende Tokens Kontextflut

21 INPUT-TOKENS

Gezieltes Lesen & Read-Dedup

CONTEXT LAYER

Dateien abschnittsweise über Line-Ranges lesen und dieselbe Datei nicht mehrfach einlesen.

nur relevante Zeilen im Kontext

19 ITERATIONEN

Deterministische Hooks

EXECUTION LAYER

Formatter, Linter und Import-Sortierung als Pre- oder Post-Hook statt als Reparaturauftrag an den Agenten.

ein Skript ersetzt einen ganzen Turn

↓

PHASE 5

Verlauf pflegen

Was nimmt der nächste Turn mit?

09 INPUT-TOKENS **TOP-HEBEL**

Observation Masking

MEMORY LAYER

Alte Tool-Outputs nach einigen Turns durch kurze Platzhalter ersetzen, etwa „pytest-Output entfernt, 3 Fehler behoben“.

ähnlich wirksam wie Zusammenfassen, aber billiger

10 INPUT-TOKENS **TOP-HEBEL**

Context Compaction

MEMORY LAYER

Verlauf kurz vor dem Limit zusammenfassen; Entscheidungen und offene Punkte in einer NOTES.md sichern.

der nächste Turn startet schlank

WARUM INPUT DOMINIERT

Jeder Turn sendet den kompletten Verlauf erneut.

In agentischen Workloads entfallen deshalb oft über 90 % der Tokens auf den Input. Output-Optimierungen wirken, aber kleiner.

Nächster Turn

gesamter Verlauf wird erneut gesendet

Drei Stellschrauben

Weniger Tokens

Input- und Output-Tokens reduzieren – der größte Hebel liegt im Kontext.

Günstigere Tokens

Caching, Routing und Batch senken den Preis, ohne den Inhalt zu ändern.

Weniger Aufrufe

Planung und Hooks vermeiden Schleifen, die den Verlauf immer wieder senden.

Prozentangaben sind typische Bandbreiten; die tatsächliche Ersparnis hängt von Workload, Modell und Anbieter ab.